Efficient Shortest Path Simplex Algorithms

Author(s): Donald Goldfarb, Jianxiu Hao and Sheng-Roan Kai

Source: *Operations Research*, Jul. - Aug., 1990, Vol. 38, No. 4 (Jul. - Aug., 1990), pp. 624-628

Published by: INFORMS

Stable URL: https://www.jstor.org/stable/171080

REFERENCES
Linked references are available on JSTOR for this article:
https://www.jstor.org/stable/171080?seq=1&cid=pdf-reference#references_tab_contents
You may need to log in to JSTOR to access the linked references.

JSTOR is a not-for-profit service that helps scholars, researchers, and students discover, use, and build upon a wide range of content in a trusted digital archive. We use information technology and tools to increase productivity and facilitate new forms of scholarship. For more information about JSTOR, please contact support@jstor.org.

Your use of the JSTOR archive indicates your acceptance of the Terms & Conditions of Use, available at https://about.jstor.org/terms

INFORMS is collaborating with JSTOR to digitize, preserve and extend access to *Operations Research*

This content downloaded from
69.166.44.12 on Mon, 22 Apr 2024 14:27:11 +00:00
All use subject to https://about.jstor.org/terms

EFFICIENT SHORTEST PATH SIMPLEX ALGORITHMS

DONALD GOLDFARB
Columbia University, New York, New York

JIANXIU HAO and SHENG-ROAN KAI
GTE Laboratories, Waltham, Massachusetts

(Received April 1988; revision received February 1989; accepted May 1989)

We consider the specialization of the primal simplex algorithm to the problem of finding a tree of directed shortest paths from a given node to all other nodes in a network of n nodes or finding a directed cycle of negative length. Two efficient variants of this shortest path simplex algorithm are analyzed and shown to require at most $(n-1)(n-2)/2$ pivots and $O(n^3)$ time.

Let $G = (N, A)$ be a finite directed graph having node-set N of cardinality n and arc-set A of cardinality m. For each arc $e \in A$, let c_e be its *length*, which may be positive, negative or zero. Given a fixed node $r \in N$, called the *root*, the shortest path (SP) problem is to find directed paths of shortest length from r to all other nodes in G or find a *negative cycle*, that is, a directed cycle of negative length.

Many methods have been proposed for solving the SP problem; for example, see the bibliographies and survey papers of Deo and Pang (1984), Dreyfus (1969), Lawler (1976, 1979) and Pierce (1975). Amongst these is the simplex method, which was first specialized for solving the SP problem by Dantzig (1957) and Minty (1958).

Although this use of the simplex algorithm is well known, the theoretical behavior of, what we shall call, the *shortest path simplex algorithm* (SPS), has not been analyzed thoroughly, and it is not generally known that certain variants of this algorithm have the same complexity bound of $O(n^3)$ as the best label-correcting algorithms for dense problems. The primary goals of this paper are to provide this analysis and to identify the most (theoretically) efficient variants of the SPS algorithm, giving sharp worst-case bounds on the number of pivots required and overall complexity bounds.

Let us first briefly review what is known. Using the flexibility that exists in the choice of the *pivot* in the simplex algorithm, Edmonds (1970) showed that it is possible to make this choice so that the number of pivots required to obtain an optimal solution grows exponentially with n. By suitably modifying Edmonds' pathological class of SP problems, Cunningham (1979) showed that the SPS algorithm with Bland's (1977) pivot rule exhibits exponential worst-case behavior.

On the positive side, it has been known for some time (e.g., see Lawler 1976) that with a properly chosen pivot rule, the SPS algorithm can solve any SP problem in at most $O(n^3)$ pivots. Orlin (1985) showed that SPS with the *most negative reduced cost pivot rule* obtains a solution (or finds a negative cycle) in at most

$$O(\min(n^2 \log \Delta,\ n^2 m \log m))$$

pivots, where Δ is an upper bound on the difference in the objective function values between any two basic feasible solutions. More recently, Akgul (1986) proposed several variants of the primal simplex algorithm that solve the SP problem, the best of which requires roughly at most $n^2/2$ pivots. These variants, however, are not straightforward implementations of the SPS algorithm as they solve the SP problem over a sequence of subgraphs.

Other adaptations of the simplex method to the SP problem include the algorithms of Florian, Nguyen and Pallottino (1981) and Roohy-Laleh (1981). The former uses the dual simplex method to resolve the shortest path problem when the *root* is changed to a different node, while the latter method is based upon a phase I simplex approach. Dial et al. (1979) and Zadeh (1979) showed that when applied to problems with all nonnegative arc distances, the SPS algorithm with the most negative reduced cost pivot rule, started from an all-artificial basis tree, can be viewed as a

Subject classifications: Networks/graphs, distance algorithms: shortest path algorithms. Programming, integer applications: simplex algorithms for shortest path problems.

0030-364X/90/3804-0624 $01.25
© 1990 Operations Research Society of America

This content downloaded from
69.166.44.12 on Mon, 22 Apr 2024 14:27:11 +00:00
All use subject to https://about.jstor.org/terms

specific implementation of Dijkstra's (1959) method, and Akgul showed that this variant requires $n - 1$ simplex pivots. Dial et al. also discussed the implementation of Bellman–Ford–Moore (BFM) (Bellman 1958, Ford 1956, Moore 1957) label-correcting type methods as primal simplex algorithms. One of these is analyzed by Akgul and shown to require at most $n(n - 1)$ pivots.

After formulating the SP problem as a linear program in the next section, we present the SPS algorithm in Section 2. In that section, we state and prove results concerning the labeling of nodes with *permanent* distance labels (dual variables) and discuss the detection of negative cycles. We analyze two pivot rules in Section 3 and show that the associated SPS variants have time complexities of $O(n^3)$. Our analysis is closely related to Cunningham's analysis of antistalling pivot rules for the primal network simplex algorithm. However, by focusing on the permanent labeling aspect of the SPS algorithm we are able to prove that these variants require at most $(n - 1) \cdot (n - 2)/2$ simplex pivots, and that this worst-case bound is tight. Other variants of the SPS algorithm and the relationship between the SPS algorithm and the Bellman–Ford–Moore label-correcting algorithm are considered in Section 4.

1. LINEAR PROGRAMMING FORMULATION OF THE SP PROBLEM

Let $t(e)$ and $h(e)$ denote, respectively, the tail and head of arc $e \in A$ and define for each node $v \in N$, the set of outward arcs, or *forward star* of v

$$F(v) \equiv \{e \in A \mid t(e) = v\}$$

and the set of inward arcs, or *backward star* of v

$$B(v) \equiv \{e \in A \mid h(e) = v\}.$$

If we associate a flow (primal variable) x_e with each arc $e \in A$ and a supply $b_r = -(n - 1)$ with node r and demands $b_v = 1$ for all other nodes $v \neq r$, then it is well known that the SP problem can be solved by applying the simplex method to the linear program

$$\begin{aligned} &\text{minimize} \quad \sum_{e \in A} c_e x_e \\ &\text{subject to} \qquad\qquad\qquad\qquad (1) \\ &\sum_{e \in B(v)} x_e - \sum_{e \in F(v)} x_e = b_v \quad \text{for all } v \in N \\ &x_e \geqslant 0 \qquad\qquad\qquad\qquad \text{for all } e \in A. \end{aligned}$$

As (1) is a transshipment problem, a specialization of the simplex method, known as the network simplex algorithm, can be used to solve it. This algorithm uses the well known fact (Dantzig 1963) that a basis in (1) corresponds to an arc-set $T \subseteq A$, where T is a spanning tree of G. On a simplex pivot an arc $e \in T$ is added to T creating a unique cycle $C(T, e)$, and an arc $f \in C(t, e) \setminus \{e\}$ is deleted yielding a new basis tree. Deleting f from T breaks T into two subtrees. Let $N_r(T, f)$ denote the set of nodes in the subtree that contains r and let $N_s(T, f) = N \setminus N_r(T, f)$.

2. THE SHORTEST PATH SIMPLEX ALGORITHM

The network simplex algorithm has some interesting properties when used to solve the shortest path problem. First, for every feasible tree T the solution is nondegenerate, i.e., $x_e > 0$ for all $e \in T$; hence the algorithm terminates in a finite number of pivots. Second, all feasible trees are *branchings rooted at r*; that is, spanning trees in which every arc is directed away from r; hence there is a directed path from r to every node $v \neq r$ and $x_e = |N_s(T, e)|$. Consequently, an optimal basis tree is a *tree of shortest paths* and the optimal node potentials (dual variables) $y_v \equiv y_v(T)$, $v \in N$, defined by requiring that the reduced costs $c_e(T) \equiv c_e + y_{t(e)} - y_{h(e)} = 0$ for all $e \in T$ and $y_r = 0$, are the desired shortest path distances. Because of the above properties if arc q is chosen to enter the basis, then the leaving arc q is the unique arc in T with $h(p) = h(q)$. Also if $t(q)$ is in $N_s(t, p)$ then G has a negative cycle. Finally, the flow values x_e, $e \in A$ are not needed in the algorithm. With these simplifications the network simplex algorithm reduces to the following.

Shortest Path Simplex (SPS) Algorithm

Step 0. Given a feasible basis tree T with associated node potentials y_v, $v \in N$.

Step 1. If $c_e(T) = c_e + y_{t(e)} - y_{h(e)} \geqslant 0$ for all $e \notin T$, stop; T is optimal. Otherwise, choose an in-arc $q \notin T$ with $c_q(T) < 0$.

Step 2. Choose the out-arc p as the (unique) arc in T with $h(p) = h(q)$. If $t(q) \in N_s(T, p)$, stop; $C(T, q)$ is a negative cycle.

Step 3. Update the node potentials and basis tree: set $y_v = y_v + c_q(T)$ for all $v \in N_s(T, p)$ and set $T \leftarrow T \cup \{q\} \setminus \{p\}$. Return to Step 1.

Clearly, the SPS algorithm terminates with either an optimal tree T^* of shortest paths or an indication that $C(T^*, q)$ is a negative cycle. In the latter case, T^* is the basis tree immediately prior to the detection of a negative cycle, which is easily accomplished since

This content downloaded from
69.166.44.12 on Mon, 22 Apr 2024 14:27:11 +00:00
All use subject to https://about.jstor.org/terms

$y_{t(q)}$ will change after a pivot if and only if $t(q) \in N_s(T, p)$. Moreover $C(T, q)$ is a negative cycle if and only if $h(q)$ is on the directed path from r to $t(q)$ and $c_q(T) < 0$.

From Step 3 of the SPS algorithm and the fact that $c_q(T) < 0$, it follows that the node potentials cannot increase, that is

Property 1. *In a simplex pivot sequence $T_0, T_1, \ldots, T^*$ $y_v(T_i) \geqslant y_v(T_j)$ for all $i < j$ and all $v \in N$.*

Definition 1. Given a feasible tree T_i, a node $v \in N$ is said to be *permanently* labeled in T_i if $y_v(T_i) = y_v(T^*)$.

From Property 1 and this definition it follows that

Property 2. *i. If u is permanently labeled in T_i but v is not, and $q \equiv (u, v) \in T^*$, then $c_q(T_i) < 0$. ii. Moreover, if G has no negative cycles then $c_q(T_i) \leqslant c_e(T_i)$ for all $e \in B(v)$.*

Proof. i. $c_q(T_i) = c_q + y_u(T_i) - y_v(T_i) = c_q + y_u(T^*) - y_v(T_i) = y_v(T^*) - y_v(T_i) < 0$. ii. Let $e \equiv (w, v) \in A$. Then $c_q(T_i) - c_e(T_i) = y_v(T^*) - y_v(T_i) - (c_e + y_w(T_i) - y_v(T_i)) \leqslant y_v(T^*) - c_e - y_w(T^*) = -c_e(T^*) \leqslant 0$.

Given a basis tree T, let the *depth* of a node v, $d_v(T)$ be defined as the number of arcs in the directed path in T from r to v, and let $S_k(T) \equiv \{v \in N \mid d_v(T) = k\}$ and $P_k(T) \equiv \{v \in N \mid d_v(T) \leqslant k\}$ denote the sets of nodes that are at depth k, and at most k, respectively, in T. Let us define T_{i_k} to be the first tree in the pivot sequence $T_0, T_1, \ldots, T^*$ generated by the SPS algorithm that contains the subtree of T^* consisting of the node-set $P_k(T^*)$. Clearly the nodes in $P_k(T^*)$ are permanently labeled in T_{i_k} and $P_k(T_{i_k}) \supseteq P_k(T^*)$. Also $i_0 = 0$. For $k \geqslant 1$, we shall call the pivot subsequence $T_{i_{k-1}+1}, T_{i_{k-1}+2}, \ldots, T_{i_k}$ stage k. Hence, the number of stages (some of which may be empty since $T_{i_{k-1}}$ may equal T_{i_k}) required by the SPS algorithm equals the maximum depth of a node in T^*.

Henceforth, we shall assume that the initial basis tree T_0 contains all arcs in the forward star $F(r)$ of the root r. Such a T_0 can be obtained by supplementing $F(r)$ by artificial arcs (r, v) with $c_{r,v} = \infty$, for all nodes v such that $(r, v) \notin A$. In this case, stage 1 is empty and the number of nonempty stages is at most $n - 2$.

3. PIVOT RULES FOR THE SPS ALGORITHM

In this section, we analyze two pivot rules, the Inward Most Negative (IMN) and the Altered IMN (AIMN) rules, which ensure that the SPS algorithm requires at most $O(n^2)$ pivots and $O(n^3)$ time. For these rules, the nodes in N are arbitrarily ordered as a circular list $v_1, v_2, \ldots, v_n$ with v_1 following v_n. In the IMN rule, after a simplex pivot is executed in which an arc e with head $h(e) = v_i$ is chosen to enter the basis tree, the next arc chosen to enter the basis is an arc q such that $h(q)$ is the first vertex v in the list $v_{i+1}, \ldots, v_n, v_1, \ldots, v_i = h(e)$ whose *backward star* $B(v)$ contains an arc with negative reduced cost and

$$c_q(T) = \min\{c_e(T) \mid e \in B(v)\} < 0. \tag{2}$$

The AIMN rule is the same as the IMN rule except that the circular list of nodes is scanned starting with node v_i, the head of the last arc to enter the basis, rather than node v_{i+1}.

Lemma 1. *If $v \in S_k(T^*)$ and v is not permanently labeled in $T_{i_{k-1}}$, v will be permanently labeled by the SPS algorithm using either the IMN or AIMN rules the first and only time that an arc in $B(v)$ enters the basis during stage k.*

Proof. When G has no negative cycles, the lemma is an immediate consequence of Property 2i and the definition of stage k. In the general case the proof is more subtle. Since v is not permanently labeled in $T_{i_{k-1}}$, Property 2i implies that some arc in $B(v)$, say $q \equiv (w, v)$, must be chosen to enter the basis during stage k. Let T_i be the basis when this first occurs. Suppose that $p \equiv (u, v) \in T^*$ and $q \neq p$; hence

$$\begin{aligned} 0 \geqslant c_q(T_i) - c_p(T_i) &= c_q + y_w(T_i) - c_p - y_u(T_i) \\ &= c_q + y_w(T_i) - y_v(T^*) \end{aligned}$$

since $y_u(T_i) = y_u(T^*)$ and $c_p + y_u(T^*) = y_v(T^*)$. But $y_w(T_{i+1}) \leqslant y_w(T_i)$ and $y_v(T_{i+1}) > y_v(T^*)$. Therefore

$$c_q(T_{i+1}) = c_q + y_w(T_{i+1}) - y_v(T_{i+1}) < 0$$

which contradicts the hypothesis that q entered the basis.

Lemma 2. (Cunningham 1979) *Let $T' = T \cup \{q\} \backslash \{p\}$ be obtained from T by the SPS algorithm with either the IMN or AIMN pivot rules. If $c_e(T') < 0$ for some $e \in B(h(q))$, then $C(T', e)$ is a negative cycle.*

Proof. From Step 3 of the SPS algorithm we have that

$$c_e(T') = \begin{cases} c_e(T) - c_q(T), & t(e) \in N_r(T, p) \\ c_e(T), & t(e) \in N_s(T, p) \end{cases}$$

since $h(e) = h(q) \in N_s(T, p)$. It also follows from (2) that $c_e(T) - c_q(T) \geqslant 0$. Therefore, $c_e(T') < 0$ implies that $t(e) \in N_s(T, p)$ and hence that $C(T', e)$ is a negative cycle.

This content downloaded from
69.166.44.12 on Mon, 22 Apr 2024 14:27:11 +00:00
All use subject to https://about.jstor.org/terms

Theorem 1. *Starting from an initial basic feasible tree T_0 that contains all arcs in $F(r)$, the SPS algorithm solves the SP problem in at most $(n-1)(n-2)/2$ pivots and $O(n^3)$ time if the IMN or AIMN rules are used.*

Proof. It follows from Lemma 2 that if the AIMN pivot rule chooses two successive in-arcs with the same head, then the SPS algorithm terminates with an indication that there is a negative cycle. Consequently, for the IMN and AIMN rules by Lemma 1 all nodes in $S_k(T^*)$ will be permanently labeled after at most $n-k$ pivots in stage k because at the start of that stage at least k nodes—those in $P_{k-1}(T^*)$—will already have been permanently labeled. Summing these bounds from $k=2$ (since stage 1 is empty) to $k=n-1$ yields the $O(n^2)$ bound on the total number of pivots.

On each simplex pivot, Steps 2 and 3—that is, checking for a negative cycle and updating the node potentials (dual variables) and the basis tree—require at most $O(n)$ operations; hence the total amount of work performed in Steps 2 and 3 is $O(n^3)$. The total amount of work performed in Step 1 is at most $O(mn)$, because in each stage the reduced cost of each arc is computed and checked at most once if the IMN rule is used, and at most twice if the AIMN rule is used. Consequently, the overall complexity of the SPS algorithm with these rules is $O(n^3)$.

By considering the linear program (1) in its totally degenerate form, i.e., $b_v = 0$ for all $v \in N$, it is obvious that the SPS algorithm is a special case of a variant of the network simplex algorithm in which all basis trees are kept strongly feasible, that is, all degenerate arcs are directed away from the root r. Hence, Cunningham's analysis of antistalling pivot rules for this variant immediately applies to the SPS algorithm, and shows that the IMN and AIMN rules require at most $(n-1)n$ pivots. These bounds are of the same order as those given in Theorem 1, but are not tight.

We now exhibit a family of SP problems upon which the SPS algorithm with the IMN or AIMN rule achieves its worst-case bound. Let $G_n = (N, A)$ be the acyclic network with $|N| = n$, $|A| = n(n-1)/2$, $(i, j) \in A$ if and only if $i > j$ and $c_{(i,j)} = -i$ for $(i, j) \in A$. Furthermore, let the nodes in N be ordered 1, 2, $\ldots$, n and let the root $r = n$. It is easily verified that

$$T_0 = \{(n, j) \mid j = 1, \ldots, n-1\}$$

and

$$T_{i_k} = \{(n-j+1, n-j) \mid j = 1, \ldots, k\} \cup \{(n-k, j) \mid j = 1, \ldots, n-k-1\}$$

and that stage k requires $n-k-1$ pivots with in-arcs $(n-k, j)$, $j = 1, 2, \ldots, n-k-1$. Observe that all arcs in A other than those in $F(r)$—there are exactly $(n-1)(n-2)/2$ such arcs—are pivoted into the tree exactly once.

4. OTHER VARIANTS OF THE SPS ALGORITHM AND THE BELLMAN-FORD-MOORE ALGORITHM

Other variants of the SPS algorithm, in addition to those that use the IMN and AIMN pivot rules, can be given that require at most $O(n^2)$ pivots and $O(n^3)$ time. For example, an $O(n^3)$ simplex variant of Yen's (1970) label correcting algorithm can be specified as follows. The node-list $v_1, v_2, \ldots, v_n$ is scanned alternatively in a forward and backward direction. During a forward scan only those arcs in $B_L(v) = \{e \in A \mid t(e) < h(e) = v\}$, that is, arcs in $B(v)$ whose tails come before v in the ordered list of nodes, are considered as candidates to enter the basis when node v is scanned. During a backward scan only those arcs in $B_G(v) = \{e \in A \mid t(e) > h(e) = v\}$ are considered. After a pivot, scanning is resumed starting from the next node in the scanning sequence as in the IMN rule. Note that both Yen's original algorithm and its simplex variants correspond to alternatively scanning columnwise the upper and lower triangles of the distance (or adjacency) matrix for the graph G.

Another $O(n^3)$ variant of the SPS algorithm corresponds to considering only those arcs whose tails are at depth k in the basis tree at the start of the kth scan, recording for each node v that is the head of one of these arcs, a single arc (u, v) that has minimum reduced cost. These arcs, which are candidates for pivoting into the basis, are then scanned and simplex pivots performed if the current reduced cost is negative. Algorithms based upon this approach are given in Goldfarb, Hao and Kai (1986).

There are variants of the SPS algorithm that require at most $O(n^2)$ pivots, yet have time complexities that are greater than $O(n^3)$. For example the Restricted Most Negative rule, first analyzed as an antistalling rule by Roohy-Laleh, has a time complexity of $O(mn^2)$ even though it can be shown to require the same number of pivots as the IMN and AIMN rules in the worst case (Goldfarb, Hao and Kai 1990).

It should be apparent from the discussion of Yen's algorithm and its simplex variant that an SPS analog can be developed for any variant of the BFM algorithm. The principal difference between such analogs is that the simplex variant updates all node potentials in $N_s(T, p)$, that is, the subtree that hangs from $h(p)$,

This content downloaded from
69.166.44.12 on Mon, 22 Apr 2024 14:27:11 +00:00
All use subject to https://about.jstor.org/terms

while the BFM variant updates only the label (node potential) of $h(p)$. Consequently, any variant of the SPS algorithm can be thought of as *label-correcting*, and it is a polynomial-time algorithm, if and only if, its BFM analog is polynomial time. However, the complexity bound for an SPS variant can be one order of n or more higher than the complexity bound for its BFM analog. For example, SPS variants that use the Least Recently Considered and Altered Outward Most Negative pivot rules require at most $O(mn)$ pivots and $O(mn^2)$ time, and the variant that uses the Outward Most Negative rule requires at most $O(n^3)$ pivots and $O(n^4)$ time (see, Goldfarb, Hao and Kai 1990) while the label-correcting analogs of these variants all require at most $O(mn)$ time.

Clearly, an SPS variant requires more work for a label-correction step than the corresponding BFM variant. But by updating many labels at once fewer complete scans are often required to solve the SP problem. As reported by Goldfarb, Hao and Kai (1986), the running times of SPS variants are comparable to their label-correcting analogs. The real advantage of SPS variants occurs when there are negative cycles in G. Although a very effective criterion for detecting a negative cycle can be given for label-correcting algorithms (see Goldfarb and Hao 1990) SPS variants still detect a negative cycle sooner. Thus, efficient variants of the SPS algorithm are preferred in applications that involve the detection of negative cycles, as in Klein's (1967) algorithm for solving minimum cost network flow problems.

ACKNOWLEDGMENT

This research was supported in part by NSF grants CDR-84-21402 and DMS 85-12277, and ONR contract N00014-87-K0214.

REFERENCES

AKGUL, M. 1986. Shortest Paths and the Simplex Method. Technical Report, Department of Computer Sciences and Operations Research Program, North Carolina State University, Raleigh, N.C.

BELLMAN, R. 1958. On a Route Problem. *Quart. of Appl. Math.* **16,** 87–90.

BLAND, R. G. 1977. New Finite Pivoting Rules for the Simplex Method. *Math. Opns. Res.* **2,** 103–107.

CUNNINGHAM, W. H. 1979. Theoretical Properties of the Network Simplex Method. *Math. Opns. Res.* **4,** 196–208.

DANTZIG, G. B. 1957. Discrete-Variable Extremum Principles. *Opns. Res.* **5,** 266–277.

DANTZIG, G. B. 1963. *Linear Programming and Extensions.* Princeton University Press, Princeton, N.J.

DEO, N., AND C. PANG. 1984. Shortest Path Algorithms: Taxonomy and Annotation. *Networks* **14,** 275–323.

DIAL, R., F. GLOVER, D. KARNEY AND D. KLINGMAN. 1979. A Computational Analysis of Alternative Algorithms and Labeling Techniques for Finding Shortest Path Trees. *Networks* **9,** 215–248.

DIJKSTRA, E. W. 1959. A Note on Two Problems in Connection With Graphs. *Numer. Math.* **1,** 269–271.

DREYFUS, S. E. 1969. An Appraisal of Some Shortest Path Algorithms. *Opns. Res.* **17,** 396–412.

EDMONDS, J. 1970. Exponential Growth of the Simplex Method for Shortest Path Problems. Unpublished Report, University of Waterloo, Waterloo, Canada.

FLORIAN, M., S. NGUYEN AND S. PALLOTTINO. 1981. A Dual Simplex Algorithm for Finding All Shortest Paths. *Networks* **11,** 367–378.

FORD, L. R. 1956. *Network Flow Theory.* The Rand Corporation Report P-923, Santa Monica, Calif.

GOLDFARB, D., AND J. HAO. 1990. Shortest Path Algorithms Using Dynamic Breadth-First Search. *Networks* (to appear).

GOLDFARB, D., J. HAO AND S. R. KAI. 1986. A Computational Comparison of Label Correcting and Simplex Algorithms for Computing Shortest Paths. Technical Report, IEOR Department, Columbia University, New York.

GOLDFARB, D., J. HAO AND S. R. KAI. 1990. Anti-Stalling Rules for the Network Simplex Algorithm. *Networks* (to appear).

KLEIN, M. 1967. A Primal Method for Minimal Cost Flow With Applications to the Assignment and Transportation Problems. *Mgmt. Sci.* **14,** 205–220.

LAWLER, E. L. 1976. *Combinatorial Optimization: Networks and Matroids.* Holt, Rinehart and Winston, New York.

LAWLER, E. L. 1979. Shortest Path and Network Algorithms. *Ann. Disc. Math.* **4,** 251–265.

MINTY, G. J. 1958. A Variant on the Shortest Route Problem. *Opns. Res.* **6,** 882.

MOORE, Z. F. 1957. The Shortest Path Through a Maze. In *Proceedings of the International Symposium on Theory of Switching,* Part II, 285–292.

ORLIN, J. B. 1985. On the Simplex Algorithm for Networks and Generalized Networks. *Math. Prog. Study* **25,** 166–178.

PIERCE, A. R. 1975. Bibliography on Algorithms for Shortest Path, Shortest Spanning Tree and Related Circuit Routing Problems. *Networks* **5,** 129–143.

ROOHY-LALEH, E. 1981. Improvements to the Theoretical Efficiency of the Simplex Method. Ph.D. Thesis, University of Carleton, Ottawa. *Dissertation Abstracts International* **13,** 448B.

YEN, J. 1970. An Algorithm for Finding Shortest Routes From All Source Nodes to a Given Destination in General Networks. *Quart. Appl. Math.* **27,** 526–530.

ZADEH, N. 1979. Near-Equivalence of Network Flow Algorithms. Technical Report 26, Stanford University, Stanford, Calif.

This content downloaded from
69.166.44.12 on Mon, 22 Apr 2024 14:27:11 +00:00
All use subject to https://about.jstor.org/terms